

JAVA Assignment

Assignment 5

Total Marks: 20

Q1. Smart Transportation System [10 Marks]

Write a Java program to design a Smart Transportation System that demonstrates inheritance, constructor chaining, method overriding, and run-time polymorphism.

Requirements:

- Create a base class Vehicle containing common information and behavior for all vehicles.
- Create the following subclasses: Car, ElectricCar, and Bike. Design an appropriate inheritance relationship among these classes.
- Each vehicle should have its own method for calculating its running cost. The method should be overridden wherever necessary to provide vehicle-specific calculations.
- The ElectricCar class should contain an additional property related to battery capacity and should initialize the inherited properties using super().
- Override an appropriate method such as displayDetails() or startEngine() so that different vehicle types exhibit different behavior.
- In the main() method, create objects of different vehicle types, store them using appropriate parent-class references, use a loop to process the vehicles, display their details and calculated running costs, and demonstrate run-time polymorphism through the overridden methods.

Test Cases:

- a. Create a normal Car and display its details and running cost.
- b. Create an ElectricCar with a given battery capacity and display its details and running cost.
- c. Create a Bike and display its details and running cost.
- d. Store a Car, ElectricCar, and Bike using Vehicle references and verify that the appropriate overridden methods are executed.
- e. Test the program with different speeds/distances or other relevant input values and verify that the running cost changes appropriately.
- f. Test a boundary case such as zero distance travelled or minimum valid input and verify that the program handles it correctly.

Q2. Employee Payroll and Bonus Management System [10 Marks]

Write a Java program to develop an Employee Payroll and Bonus Management System that demonstrates abstraction, inheritance, method overriding, method overloading, and run-time polymorphism.

Requirements:

- Create an abstract class Employee containing common employee information and an abstract method for calculating salary.
- Create the following subclasses: FullTimeEmployee, PartTimeEmployee, and ContractEmployee.
- Each employee type must calculate its salary differently by overriding the salary calculation method.
- Implement overloaded calculateBonus() methods in the Employee class: one method should calculate a fixed bonus, another should accept a percentage and calculate the bonus based on salary, and another overloaded version should accept an additional parameter such as a performance rating.
- In the main() method, create different employee objects using Employee references, store them in an array or collection, use a loop to calculate and display their salaries, demonstrate the overloaded bonus methods, and demonstrate run-time polymorphism through salary calculation.

Test Cases:

- a. Create a FullTimeEmployee and verify the salary calculation.
- b. Create a PartTimeEmployee with different working hours and hourly rates and verify the salary.
- c. Create a ContractEmployee and verify its salary calculation.
- d. Store all three employee types using Employee references and verify that the correct calculateSalary() method is executed.
- e. Calculate the bonus using the fixed-bonus version of calculateBonus().
- f. Calculate the bonus using the percentage-based and performance-based overloaded versions and verify the results.